

Practical 3:

* Problem statement:

→ WAP to implement stack using array. Demonstrate the following operations on stack: Push(), Pop(), Peek() and display().

* Algorithm

1. Push operation

1. Start
2. Check if $top == MAX - 1$.
3. If true, display "Stack Overflow".
4. Otherwise, increment top .
5. Store the value at $Stack[top]$.
6. Display "Value Pushed".
7. Stop.

2. Pop operation

1. Start
2. Check if $top == -1$.
3. If true, display "Stack underflow".
4. Otherwise, display $Stack[top]$.
5. Decrement top .
6. Stop.

3. Peek operation

1. Start
2. Check if $top == -1$.
3. If true, display "stack is empty".
4. Otherwise, display $Stack[top]$ as the top element.
5. Stop.

4. Display operation

1. Start
2. Check the value of top .
3. Traverse the stack from top to 0.
4. Display each element.
5. Stop.

For main:

Push 10

Push 20

Push 30

Display Stack $\rightarrow 30, 20, 10$

Peek $\rightarrow 30$

Pop $\rightarrow 30$

Display Stack $\rightarrow 20, 10$


```
#include <iostream>
using namespace std;
```

```
#define MAX 5
```

```
int stack[MAX];
int top = -1;
```

```
void Push (int value) {
```

```
    if (top == MAX-1),
```

```
        cout << "Stack overflow\n";
```

```
    else {
```

```
        stack[++top] = value;
```

```
        cout << value << "Pushed\n";
```

```
    }
```

```
}
```

```
void Pop() {
```

```
    if (top == -1) {
```

```
        cout << "Stack underflow\n";
```

```
    } else {
```

```
        cout << stack[top--] << "Popped\n";
```

```
    }
```

```
}
```

```

void peek() {
    if (top == -1) {
        cout << "stack is empty\n";
    }
    else {
        cout << "TOP element: " <<
            arr[top] << endl;
    }
}

```

```

void display() {
    cout << "stack: ";
    for (int i = top; i >= 0; i--)
        cout << arr[i] << " ";
    cout << endl;
}

```

```

int main() {
    push(10);
    push(20);
    push(30);
}

```

```

display();

```

```

peek();

```

```

pop();

```

```

display();

```

```

return 0;
}

```

Output

10 Pushed

20 Pushed

30 Pushed

Stack : 30 20 10

Top element : 30

30 Popped

Stack : 20 10